PES University — Department of CSE
Software Engineering — Laboratory

Final Lab Report

Lab 1 — Requirements Engineering & UML Use-Case Modelling • Lab 3 — Component Modelling & Architectural Pattern Selection

Name	Pranaav P
SRN	PES1UG24AM387
Topic	Urban Rooftop Gardening Co-op App (Problem Statement #65)
Lab 3 applies the	same topic to component modelling & architecture

1. Lab 1 — Requirements Engineering & UML Use-Case Modelling

1.1 Problem Overview

A community urban-agriculture app for registering rooftop plots, logging crop yields and harvest schedules, sharing soil-maintenance tips, and trading surplus produce. Actors: Co-op Gardener and Community Leader.

1.2 Requirements Table

ID	Type	Priority	Description	Acceptance Criteria	Rationale
FR-001	Functional	High	Allow a Co-op Gardener to log seasonal crop planting dates for a registered plot, and compute the estimated harvest window and expected yield weight from crop type and planting date.	Pass: harvest window/alert generated within the predicted window for a valid record. Fail: an invalid planting-date sequence (harvest-before-planting) is accepted.	Core value of the app — plan harvests and anticipate yields.
FR-002	Functional	High	Allow a Co-op Gardener to register a rooftop plot (location, area m², sunlight, owner) and assign each a unique Plot ID.	Pass: plot with all mandatory fields saved and retrievable by Plot ID. Fail: saved with a missing field or duplicate Plot ID.	Plots are the anchor entity for crop logs, yields and trades.
FR-003	Functional	Medium	Allow a gardener to post surplus produce (item, quantity, availability date) to a community trade board and let others reserve/claim	Pass: posted listing visible to others; status → 'Reserved' when claimed. Fail: a claimed listing stays available to others.	Reduces food waste; supports produce sharing.

			listings.		
FR-004	Functional	Medium	Allow members to publish and browse soil-maintenance tips, tagged by topic (composting, pH, pests), newest first.	Pass: published tip appears in its tagged feed within one refresh. Fail: a tip is published without a topic tag.	Builds shared community knowledge.
FR-005	Functional	Low	Allow a Community Leader to broadcast announcements to all members and moderate (hide/remove) rule-violating trade-board listings.	Pass: broadcast reaches all active members; moderated listing removed from the public board. Fail: a non-leader can broadcast or moderate.	Governance; keeps the shared boards trustworthy.
NFR-001	Non-Functional (Performance & Security)	High	The trade board updates surplus listings in near real-time (≤ 2 s) across members, over authenticated, encrypted (TLS) connections.	Pass: benchmarking confirms ≤ 2 s median update latency and TLS-only transport under simulated peak load (200 concurrent members). Fail: latency exceeded or any unencrypted update observed.	Freshness prevents double-claims; encryption protects member data.
NFR-002	Non-Functional (Usability & Availability)	Medium	The app is usable outdoors (WCAG 2.1 AA contrast, touch targets ≥ 44 px) and the backend keeps ≥ 99.5% monthly availability in the growing season.	Pass: accessibility audit shows no AA contrast failures and uptime ≥ 99.5%. Fail: AA failures found or uptime < 99.5%.	Used outdoors in bright light; reliability keeps logging/trading dependable.

Table 1 — 5 Functional + 2 Non-Functional requirements.

1.3 UML Use-Case Diagram

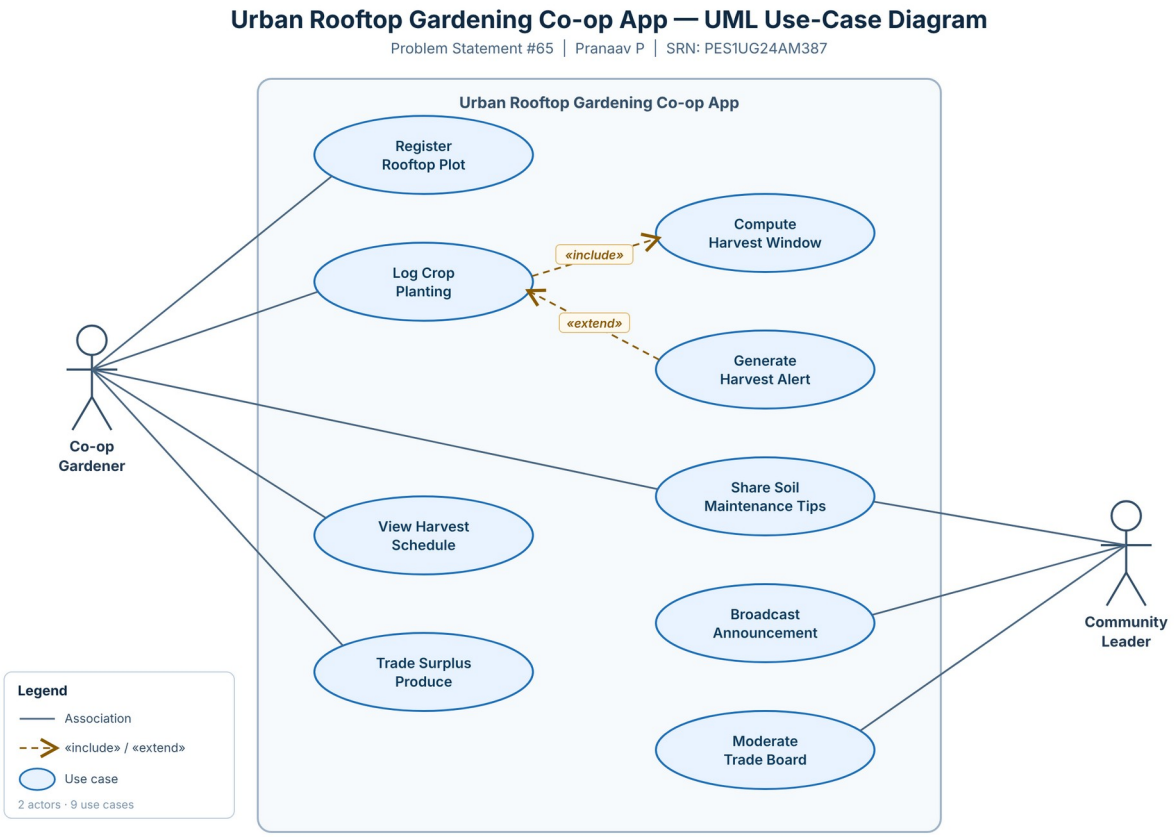


Figure 1 — Use-case diagram. «include»: Log Crop Planting → Compute Harvest Window; «extend»: Generate Harvest Alert → Log Crop Planting.

1.4 Use-Case Flow Specification — Log Crop Planting (UC-02)

Primary actor: Co-op Gardener.

Preconditions: logged in; has at least one registered plot (FR-002); crop catalogue available.

Postconditions: planting record persisted; harvest window + yield computed and stored («include»); harvest alert optionally scheduled («extend»).

Main Success Scenario:

1. Gardener selects a registered plot.
2. System displays the planting form and crop catalogue.
3. Gardener selects crop type and enters the planting date.
4. System validates the planting date (valid, not an impossible sequence).
5. System computes the harvest window and expected yield from growth duration and plot area [«include» Compute Harvest Window].
6. System saves the planting record with the computed window and yield estimate.
7. System confirms and displays the estimated harvest window.
   - **7a.** [«extend» Generate Harvest Alert] If alerts are enabled, schedule a harvest-window notification.

Alternate Flow — A1: Invalid planting date sequence (at step 4):

- **4a.1** System detects an invalid date (earlier than the plot's last harvest, or harvest-before-planting).
- **4a.2** System rejects the entry and shows a validation message.

4a.3 System prompts for a valid date and returns to step 3.

4a.4 If the gardener cancels, no record is saved (the invalid sequence is never accepted).

2. Lab 3 — Component Modelling & Architectural Pattern Selection

2.1 Scenario Overview

The Urban Rooftop Gardening Co-op App is a multi-user community platform. Many gardeners and community leaders, each on their own phone, register rooftop plots, log crop plantings, trade surplus produce on a live board, and share soil-maintenance tips. It needs one shared, consistent data store, near real-time trade-board updates (≤ 2 s), encrypted transport, and centralised moderation / broadcasts by community leaders.

2.2 Architectural Style Analysis

Style	Structure	Pros (for this scenario)	Cons (for this scenario)	Verdict
Client-Server	Thin mobile clients + central backend server and shared database.	One authoritative, consistent data source for all users; central auth & moderation; server can push real-time trade-board updates. Natural fit for a multi-user app.	Server is a single point of failure / bottleneck — mitigated by replicating and load-balancing the server tier.	**Chosen**
Microservices	Small, independently deployable services.	Independent scaling and fault isolation of the busy trade board.	Operational complexity, network latency and data-consistency challenges — disproportionate for a community/student-scale app.	**Rejected**
Layered	Presentation / Business / Data layers inside one app.	Clean separation of concerns; simple to build and test.	Describes a single application's internal structure — it cannot, on its own, share data across many users' devices.	**Rejected**

Table 2 — Candidate architectural styles evaluated against the Rooftop Gardening app scenario.

2.3 Chosen Architecture & Components

Client-Server (multi-tier). Five components are identified (Plot & Harvest and Trade & Community map to the given Order-Manager / Service roles, plus the remaining components):

- Gardener / Leader Mobile Client (Client tier) — the thin mobile UI for all user actions.
- API Gateway – Auth & Routing (Server tier) — single entry point; authenticates and routes requests.
- Plot & Harvest Service (Server tier) — plot registration, crop logging, harvest-window & yield computation (FR-001, FR-002).
- Trade & Community Service (Server tier) — real-time trade board, soil tips, and leader broadcasts / moderation (FR-003–FR-005).
- Data Repository (Data tier) — one shared database for plots, crops, listings, tips and users.

2.4 UML Component Diagram

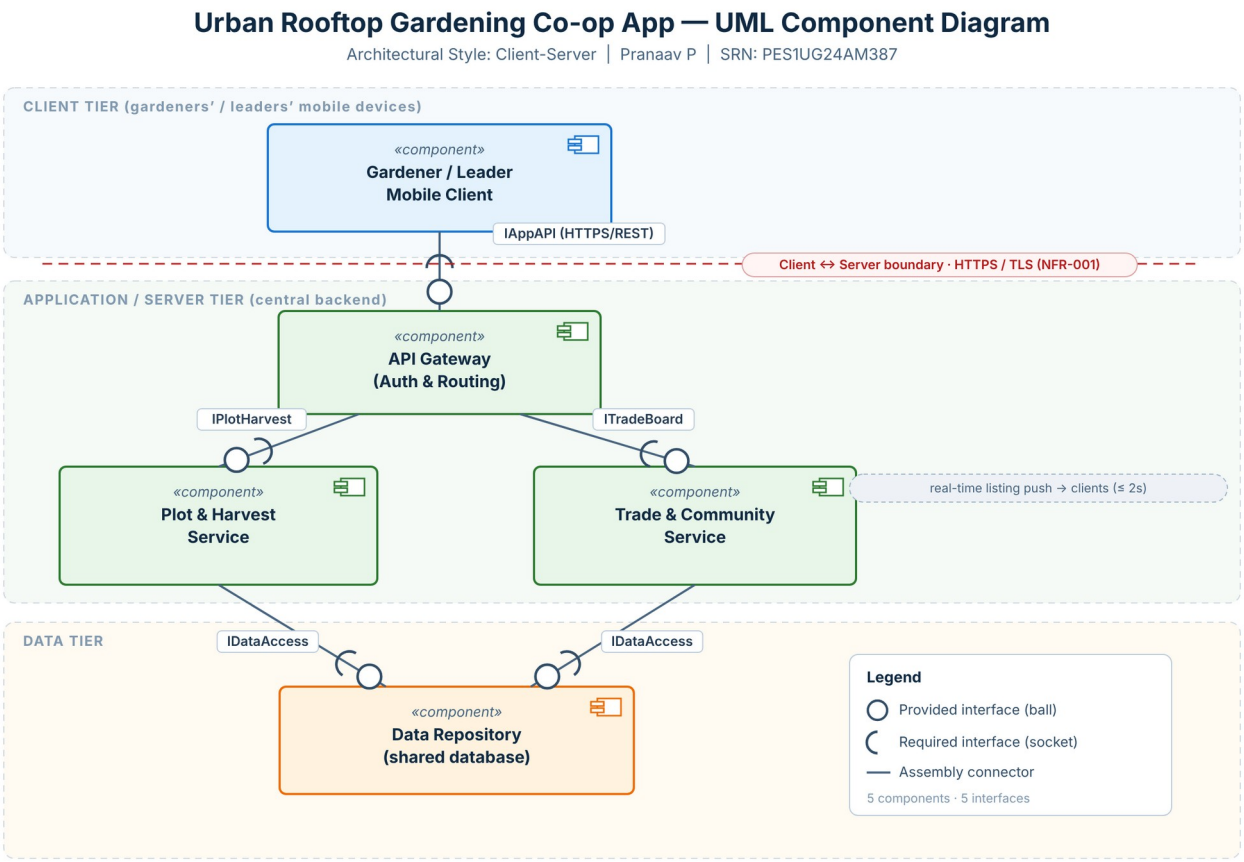


Figure 2 — Component diagram: 5 components across client / server / data tiers, with 5 provided/required (ball-and-socket) interfaces (IAppAPI, IPlotHarvest, ITradeBoard, and IDataAccess shared by two services). The client ↔ server boundary is secured with HTTPS/TLS.

2.5 Written Justification

We chose a Client-Server Architecture for the Urban Rooftop Gardening Co-op App.

Reason 1 — Shared, consistent data for many users: many gardeners and leaders, each on their own phone, must see and act on the same shared data (plots, listings, soil tips). A central server with thin clients gives one authoritative, consistent data source — essential for the trade board, where two gardeners must not both claim the same produce. A single-device Layered app cannot share data across users, and full Microservices would add complexity out of proportion to a community app.

Reason 2 — Centralised control & real-time updates: the requirements need centralised governance and live updates — a Community Leader broadcasts and moderates from one place (FR-005), and the server pushes trade-board changes to every client to meet NFR-001's ≤ 2 s freshness target. A central server is the natural home for that control and real-time fan-out.

Security advantage: all member data and trade transactions flow through one hardened backend over authenticated, encrypted (TLS) connections (NFR-001). Authentication and role-based authorisation are enforced centrally at the API Gateway (e.g. only leaders can moderate), so security policy lives in one auditable place and sensitive data never resides permanently on the thin clients.

Performance benefit: the central server maintains push channels (WebSockets / subscriptions) to deliver trade-board changes within the ≤ 2 s target and caches/indexes frequently-read data server-side for fast queries; thin

clients keep the phone-side light and responsive outdoors (NFR-002). The single-server availability risk is mitigated by replicating and load-balancing the server tier.